

Technical Requirements — DICOM 3D Brain Viewer

Status: **implemented and verified**, 2026-09-23. Every requirement below is traced to the code that satisfies it and the test that proves it.

1. Scope

Browser-based 3D viewer for brain MR studies. Local single-user deployment.

In: DICOMweb backend over a local file store; study/series browsing; drag-and-drop upload of files, folders or `.zip` ; 2×2 viewer (axial / sagittal / coronal MPR with synchronised crosshairs + GPU volume rendering); brain-oriented window and transfer-function presets.

Out: measurements, annotations, segmentation, surface rendering, hanging protocols, authentication, multi-user, hosting, STOW-RS, PHI handling beyond "samples are anonymised".

2. Platform

Backend	Python 3.12 · FastAPI 0.141 · pydicom 3 · numpy 2 · SQLite (stdlib) · uvicorn on :8001 (factory mode)
Frontend	Node 24 · React 19 · Vite 8 · TypeScript 6 (strict) · Tailwind 4 · Cornerstone3D 5.10 · TanStack Query 5 · react-router 8
Test	pytest · ruff · mypy --strict · vitest + MSW · Playwright
Browser	WebGL2 required; detected at startup with an explicit fallback page

3. Functional requirements

#	Requirement	Implementation	Verified by
F1	Ingest a DICOM file, reject non-DICOM and non-image objects with a per-file reason	<code>ingest/reader.py</code>	<code>test_reader.py</code> , <code>test_indexer.py</code>
F2	Reject syntactically invalid SOP/Series/Study UIDs before they reach the filesystem	<code>ingest/reader.py</code> + <code>store._ensure_within</code>	<code>test_reader.py</code> , <code>test_store.py</code>
F3	Decode JPEG 2000 / JPEG-LS / RLE once at ingest to Explicit VR LE, preserving SOP Instance UID	<code>ingest/decode.py</code>	<code>test_decode.py</code>
F4	Order slices anatomically by <code>dot(ImagePositionPatient, normal)</code> ; fall back to InstanceNumber then filename	<code>geometry.sort_instances</code>	<code>test_geometry.py</code>
F5	Classify a series as a volume only if geometry-sorted, ≥ 3 slices, consistent dimensions, single orientation, uniform spacing — with a reason on failure	<code>geometry.volume_info</code>	<code>test_geometry.py</code> (one case per reason)
F6	Serve QIDO-RS study/series/instance search with filters and paging	<code>dicomweb/qido.py</code>	<code>test_qido.py</code>
F7	Serve WADO-RS metadata (sorted, X-Sort-Method), <code>frames/{n}</code> as multipart/related , instance retrieval, rendered PNG	<code>dicomweb/wado.py</code>	<code>test_wado.py</code>
F8	Emit DICOM JSON per PS3.18 Annex F (PN components, empty elements, bulk omission, numeric typing, sequences)	<code>dicomweb/json_model.py</code>	<code>test_json_model.py</code> (cross-checked against pydicom)
F9	Expose volume geometry for the client: dims, spacing, origin, direction, estimated bytes	<code>api/volume.py</code>	<code>test_volume_api.py</code>
F10	Accept multi-file and <code>.zip</code> upload; partial success; reject path traversal and zip bombs	<code>api/upload.py</code>	<code>test_upload.py</code>
F11	Browse studies → series with thumbnails; non-volume series greyed with reason	<code>components/browser/*</code>	<code>SeriesGrid.test.tsx</code>
F12	Upload by drag-and-drop of files, a folder, or a <code>.zip</code>	<code>UploadDropzone.tsx</code>	<code>UploadDropzone.test.tsx</code>
F13	Open a volume into 3 MPR viewports + 1 3D viewport sharing one GPU volume	<code>cornerstone/viewports.ts</code> , <code>ViewerLayout.tsx</code>	e2e + manual
F14	Synchronised crosshairs; per-plane scroll; window/level; zoom; pan; presets; reset	<code>cornerstone/toolGroups.ts</code> , <code>Toolbar.tsx</code>	e2e, <code>Toolbar.test.tsx</code>
F15	Report load progress; refuse non-volume series; confirm above 1 GB	<code>hooks/useVolume.ts</code>	<code>useVolume.test.tsx</code>

4. Non-functional requirements

#	Requirement	How it is met
N1	A failed frame must never be presented as valid image data	<code>loadVolume</code> rejects on the first <code>IMAGE_LOAD_ERROR</code> , releases the partial volume, names the slice
N2	Concurrent frame requests must not corrupt the index	Connection per request (<code>get_db</code>), WAL, <code>busy_timeout</code> ; a shared connection raised <code>InterfaceError</code> under load
N3	Uploads must not escape the store	UID syntax validation, <code>_ensure_within</code> containment on both file writes and zip entries
N4	Uploads must not exhaust disk or memory	Chunked streaming read with an early <code>413</code> ; decompressed-byte budget shared with the raw request limit
N5	Opening, leaving and reopening series must not leak	Abort in-flight load, destroy tool groups, disable viewports, release the volume; latch keyed to <code>seriesUid</code>
N6	Frame retrieval must not decode	Guaranteed by F3; <code>frames/{n}</code> is a byte slice with <code>Content-Length</code> and cache headers
N7	Type and lint discipline	<code>mypy --strict</code> on <code>app/</code> , TypeScript <code>strict</code> , <code>ruff</code> , <code>oxlint</code> , <code>Prettier</code> — all clean
N8	Test output must be pristine	Zero warnings in both suites; third-party deprecations filtered by exact match, never blanket

5. Interfaces

```

GET /api/health → {"status":"ok"}
GET /api/series/{series_uid}/volume-info → {seriesUid,isVolume,reason,dims,spacing,origin,
                                         direction,modality,sortMethod,instanceCount,estimatedBytes}

POST /api/upload (multipart, field `files`, repeated)
               → {accepted,skipped:[{file,reason}],studyUids}

GET /dicomweb/studies[?PatientName&PatientID&StudyDate&limit&offset] → application/dicom+json
GET /dicomweb/studies/{study}/series → application/dicom+json
GET /dicomweb/studies/{study}/series/{series}/instances → application/dicom+json
GET /dicomweb/studies/{study}/series/{series}/metadata → application/dicom+json, X-Sort-Method
GET /dicomweb/studies/{study}/series/{series}/instances/{sop} → multipart/related; type="application/dicom"
GET /dicomweb/studies/{study}/series/{series}/instances/{sop}/frames/{n} → multipart/related; type="application/octet-stream"
GET /dicomweb/studies/{study}/series/{series}/instances/{sop}/rendered?viewport=W,H → image/png

```

Errors: `400` unsafe zip entry · `404` unknown UID or frame out of range · `413` over the size budget · `422` malformed query parameter. Bodies are `{detail}`.

Storage: `data/store/<StudyUID>/<SeriesUID>/<SOPUID>.dcm` plus `data/store/index.sqlite` (`study` · `series` · `instance`). Both git-ignored.

6. Verification

`scripts/test.ps1` → **83 backend tests** (pytest) + 3 script tests + `ruff` + `mypy` + **37 frontend tests** (vitest), exit 0, no warnings. `cd frontend; npm run e2e` → **1 Playwright test**: seeds a synthetic 40-slice series, opens it, asserts four non-blank canvases, scroll, crosshair sync, preset change and reopen. No network, no sample data.

Sample data: two brain MR series from TCIA UPENN-GBM patient `UPENN-GBM-00041` — T1 MPRAGE (160 slices, transcoded to JPEG 2000 Lossless so the decode path runs on real data) and T2-FLAIR (60 slices, uncompressed). CC BY 4.0, fetched and verified by `scripts/fetch_samples.py` against `scripts/samples.json`. Nothing binary is committed.

7. Known limitations

- Multi-frame instances are indexed but never classified as volumes; per-frame spacing would have to come from the enhanced functional-group sequences.
- Cornerstone-facing code is unit-tested against a hand-written mock of Cornerstone; only the Playwright test observes real library event shapes.
- No cancellation of the info/metadata HTTP requests — React Query drops the response but the request completes.

- `npm audit` reports transitive advisories under `@cornerstonejs` → `vtk.js` with no non-breaking upstream fix; the app is local-only.